Introduction au SQL

Séance 2 : Manipuler les relations entre tables : jointures, conditions et requêtes multi-tables

Fatimetou ZEINE

06,08 et 10 Octobre 2025

Rappel – Séance 1

Fonction

SQL

SQL (Structured Query Language) est le langage standard pour interagir avec les bases de données relationnelles

Commandes principales

- **CREATE DATABASE / TABLE** : créer une base ou une table.
- **SELECT** : interroger des données.
- **INSERT INTO** : ajouter des lignes.
- **UPDATE** : modifier des données.
- **DELETE** : supprimer des lignes.
- **DROP** : supprimer une base ou une table.

Types de données

- **INT, FLOAT, DECIMAL** : nombres entiers et décimaux.
- **CHAR, VARCHAR, TEXT** : chaînes de caractères.
- **DATE, TIME, DATETIME** : types temporels.
- **BOOLEAN** : valeurs vraies/fausses (0 ou 1 selon les SGBD).

Opérations CRUD

- C - Create : **INSERT INTO**
- R - Read : **SELECT**
- U - Update : **UPDATE**
- D - Delete : **DELETE**

Clauses SQL utiles

- **WHERE** : filtre les lignes selon une condition.
- **GROUP BY** : regroupe les résultats par valeurs identiques.
- **ORDER BY** : trie les résultats.
- **HAVING** : filtre après regroupement.
- **LIMIT** : limite le nombre de lignes retournées.
- **DISTINCT** : élimine les doublons.

Fonctions d'agrégation

- **COUNT()** : compte le nombre de lignes.
- **SUM()** : calcule la somme.
- **AVG()** : calcule la moyenne.
- **MIN() / MAX()** : valeurs minimale et maximale.

Contraintes de données

- **NOT NULL** : interdit les valeurs nulles.
- **PRIMARY KEY** : identifie une ligne de manière unique.
- **FOREIGN KEY** : lie une table à une autre.

Operateurs

- **AND / OR** : combiner plusieurs conditions.
- **LIKE** : recherche avec motifs (joker %, _).
- **IN** : vérifie si une valeur appartient à une liste.
- **NOT** : inverse une condition. - **!=** ou <> : opérateur de différence (not equal).
- **IS NULL** : teste la nullité d'une valeur.
- **BETWEEN** : teste une valeur dans un intervalle.

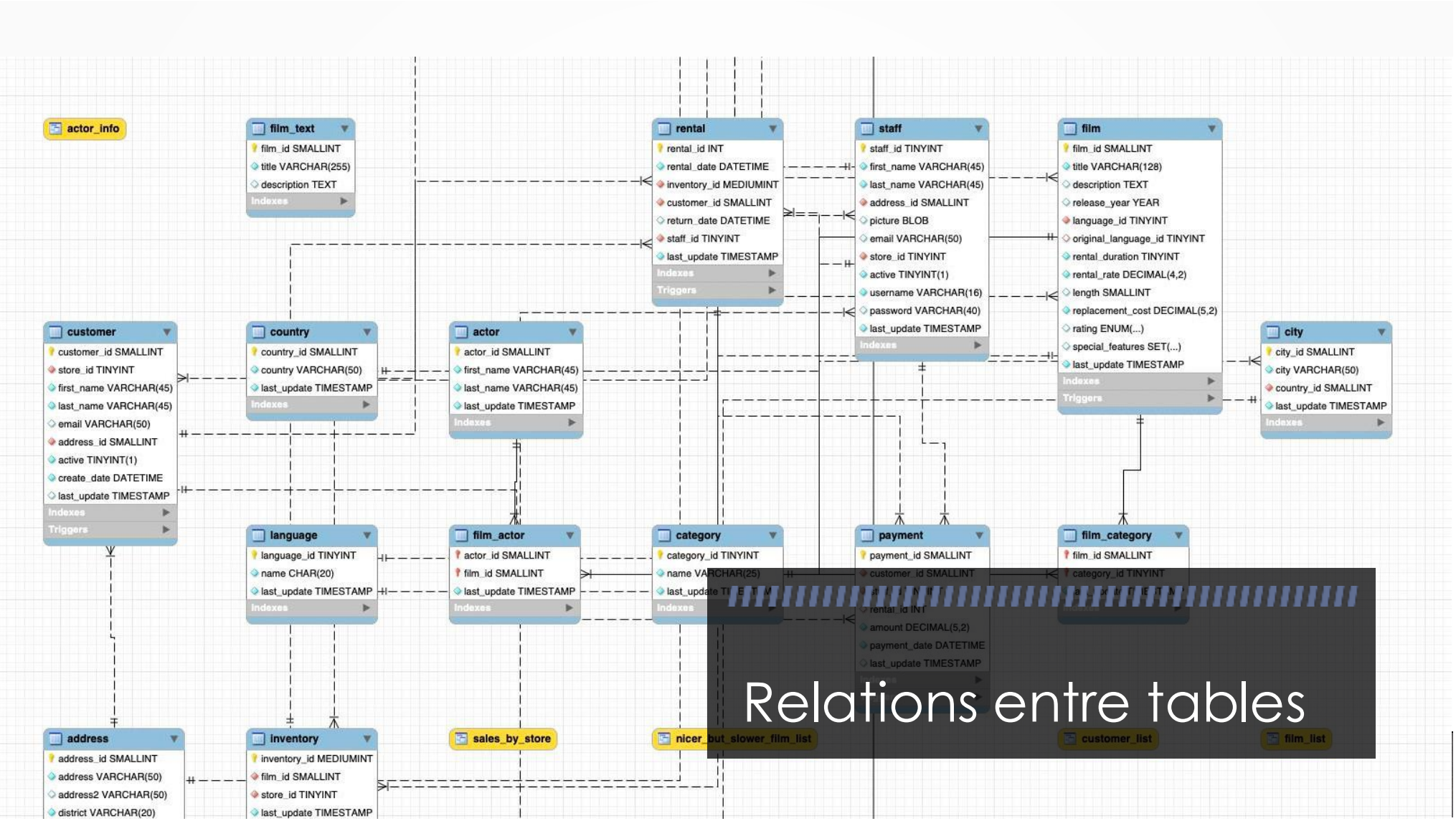
Operateur CASE / WHEN

- CASE permet d'effectuer des **conditions** dans une requête SQL.
- Il joue le rôle d'un **IF… ELSE**.
- Il est souvent utilisé pour **créer des colonnes dynamiques ou catégoriser** des données**.**

```
CASE
  WHEN condition1 THEN résultat1
  WHEN condition2 THEN résultat2
  ...
  ELSE résultat_par_defaut
END
```

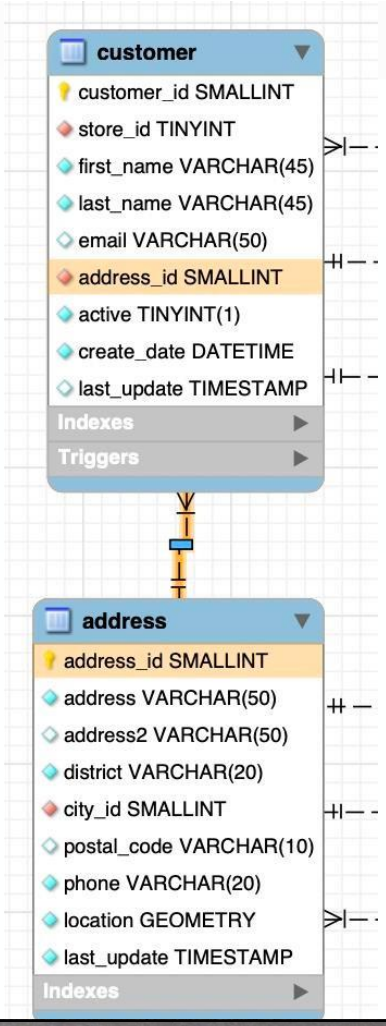
```
SELECT title, length,
      CASE
        WHEN length >= 120 THEN 'Long métrage'
        WHEN length BETWEEN 90 AND 119 THEN 'Film standard'
        ELSE 'Court métrage'
      END AS type_film
FROM film;
```

Correction du TP1



Relations entre tables

Clé primaire (PRIMARY KEY) et Clé étrangère (FOREIGN KEY)



INNER JOIN – Jointure interne

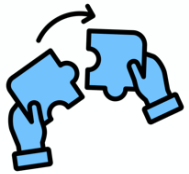
Ne conserve que les lignes **qui ont des correspondances dans les deux tables**.

```
SELECT
    *
FROM
    film
        INNER JOIN
    film_category ON film.film_id = film_category.film_id;
```

Principes de conception derrière la séparation en plusieurs tables

Normalisation

Modularité



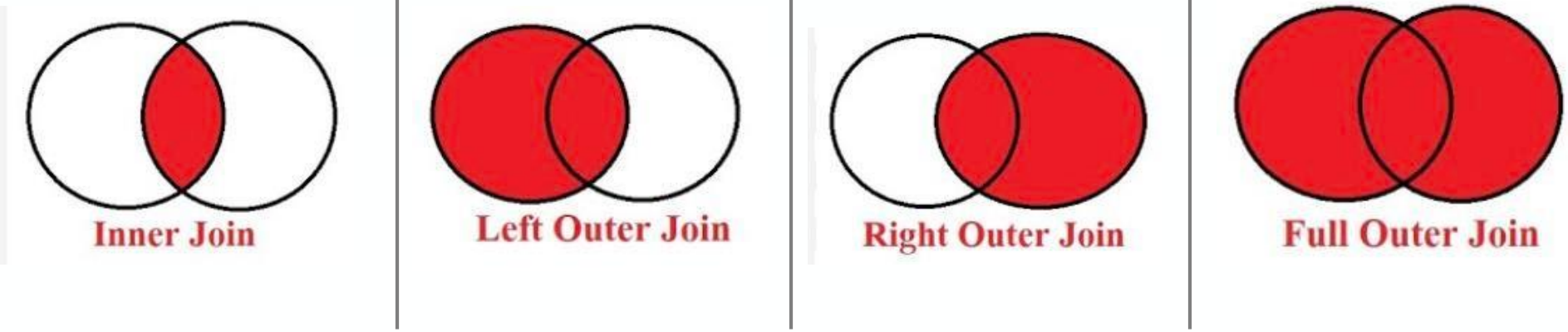
Cohérence

Performance

Évolutivité

Les jointures

- Une **jointure** permet de combiner les données de **plusieurs tables** en une seule vue.
- Elle relie les lignes selon des **clés communes** (souvent clé primaire <-> clé étrangère).



LEFT JOIN / RIGHT JOIN – Jointures externes

LEFT JOIN : conserve **tous les enregistrements de la table de gauche**, même sans correspondance à droite.

```
SELECT
    c.customer_id, c.first_name, c.last_name, r.rental_date
FROM
    customer c
        LEFT JOIN
    rental r ON c.customer_id = r.customer_id;
```

Jointure intersection (INNER JOIN)

```
SELECT *
FROM table1
INNER JOIN table2
ON table1.key = table2.key;
```

INNER JOIN

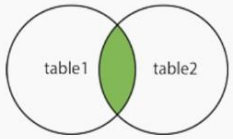


Table 1

Produit	Prix
Kiwis	6$
Bananes	9$
Tomates	12$
Fraises	7$

Table 2

Produit	Quantité
Kiwis	12
Bananes	20
Tomates	5
Brocolis	9

Table 1 INNER JOIN Table 2

Table fusionnée

Produit	Prix	Quantité
Kiwis	6$	12
Bananes	9$	20
Tomates	12$	5

Jointure à gauche (LEFT JOIN),

```
SELECT *
FROM table1
LEFT JOIN table2
ON table1.key = table2.key
```

LEFT JOIN

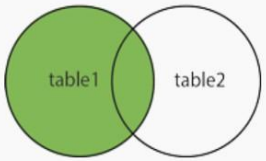


Table 1

Produit	Prix
Kiwis	6$
Bananes	9$
Tomates	12$

Table 2

Produit	Quantité
Kiwis	12
Bananes	20
Tomates	5

Table 1 LEFT JOIN Table 2

Table fusionnée

Produit	Prix	Quantité
Kiwis	6$	12
Bananes	9$	20
Tomates	12$	5
Fraises	7$	NULL

Jointure externe (OUTER JOIN)

```
SELECT *
FROM table1
FULL JOIN table2
ON table1.key = table2.key
```

FULL OUTER JOIN

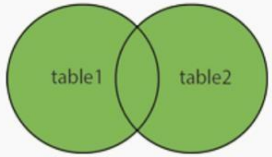


Table 1

Produit	Prix
Kiwis	6$
Bananes	9$
Tomates	12$
Fraises	7$

Table 2

Produit	Quantité
Kiwis	12
Bananes	20
Tomates	5
Brocolis	9

Table 1 FULL JOIN Table 2

Produit	Prix	Quantité
Kiwis	6$	12
Bananes	9$	20
Tomates	12$	5
Fraises	7$	NULL
Brocolis	NULL	9

TD 2 (1)

1. Afficher les titres de films appartenant à la catégorie "Comedy"

Utiliser des jointures entre les tables film, film_category et category.
L'objectif est d'afficher les titres (title) des films dont la catégorie (name) est **Comedy**.

2. Lister tous les clients avec leur nombre de locations

Utiliser une jointure entre les tables customer et rental, puis grouper les résultats par client.
Afficher le prénom, le nom et le **nombre total de locations** effectuées par chaque client.

3. Afficher les paiements avec le nom du client et celui de l'employé

Utiliser des jointures entre payment, customer et staff pour afficher les colonnes suivantes :

- le nom de famille du client (customer.last_name),
- le nom de l'employé responsable (staff.last_name),
- et le montant du paiement (amount).

TD 2 (2)

4. Compter les films selon leur durée avec CASE WHEN

Écrire une requête affichant, pour chaque **catégorie de durée**, le **nombre de films**.
Utiliser CASE pour regrouper les films dans trois catégories :
•"Court" si la durée est inférieure à 90 minutes,
•"Standard" si elle est entre 90 et 120 minutes inclus,
•"Long" si elle dépasse 120 minutes.
Afficher deux colonnes :
•type_duree (catégorie issue de CASE),
•et le nombre de films par type.
Utiliser GROUP BY.

5. Lister les films qui n'ont jamais été loués

Réaliser une jointure LEFT JOIN entre film, inventory, et rental.
Identifier les films pour lesquels aucune location n'a été enregistrée (rental.rental_id IS NULL).
Afficher uniquement le titre (title) des films concernés.

```sql
-- Question 1 : Afficher les titres de films appartenant à la catégorie "Comedy"

SELECT
   f.title
FROM
   film f
      INNER JOIN
   film_category fc ON f.film_id = fc.film_id
      INNER JOIN
   category c ON fc.category_id = c.category_id
WHERE
   c.name = 'Comedy'

-- Question 2 : Lister tous les clients avec leur nombre de locations
SELECT
   c.first_name,
   c.last_name,
   COUNT(r.rental_id) AS nombre_locations
FROM
   customer c
      LEFT JOIN
   rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id , c.first_name , c.last_name
ORDER BY nombre_locations DESC
```

```
-- Question 3 : Afficher les paiements avec le nom du client et celui de l'employé

SELECT
   c.last_name AS client_name,
   s.last_name AS staff_name,
   p.amount
FROM
   payment p
      JOIN
   customer c ON p.customer_id = c.customer_id
      JOIN
   staff s ON p.staff_id = s.staff_id

--Question 4 : Compter les films selon leur durée avec CASE WHEN

SELECT
   CASE
      WHEN length < 90 THEN 'Court'
      WHEN length BETWEEN 90 AND 120 THEN 'Standard'
      ELSE 'Long'
   END AS type_duree,
   COUNT(*) AS nombre_films
FROM film
GROUP BY type_duree;
```

```
-- Question 5 : Lister les films qui n'ont jamais été loués

SELECT f.title
FROM film f
LEFT JOIN inventory i ON f.film_id = i.film_id
LEFT JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.rental_id IS NULL;

-- Question 6 : Créer une colonne conditionnelle pour classer les paiements

SELECT c.first_name, c.last_name,
    p.amount,
    CASE
        WHEN p.amount < 2 THEN 'Faible'
        WHEN p.amount BETWEEN 2 AND 5 THEN 'Moyen'
        ELSE 'Élevé'
    END AS niveau_paiement
FROM payment p
JOIN customer c ON p.customer_id = c.customer_id;
```

Quiz 2– Requêtes, jointures et détection d'erreurs

Ce quiz couvre les notions essentielles de SQL abordées en séance 2 : *requêtes de base, filtres, fonctions d'agrégation, jointures, opérateurs, conditions CASE WHEN*, ainsi que la détection d'erreurs fréquentes et la validation de la syntaxe.

fatimetouzeyn47@gmail.com Switch account

Not shared

Next

Clear form

https://forms.gle/waozQN3VNpLj95DR7

TP 2

Question 1: Affiche la liste des clients avec leur prénom, nom de famille et le nombre total de films qu'ils ont loués. Trie les résultats du plus grand au plus petit nombre de locations.

Question 2: Affiche les films accompagnés du nom de leur catégorie et du nombre total de fois où ils ont été loués.

Question 3: Affiche tous les clients, même ceux qui n'ont jamais loué de film. Le nombre de locations doit apparaître comme zéro s'il n'y en a aucune.

séance 3 => (sous-requêtes, fonctions, mini- project…)